

Program Reasoning

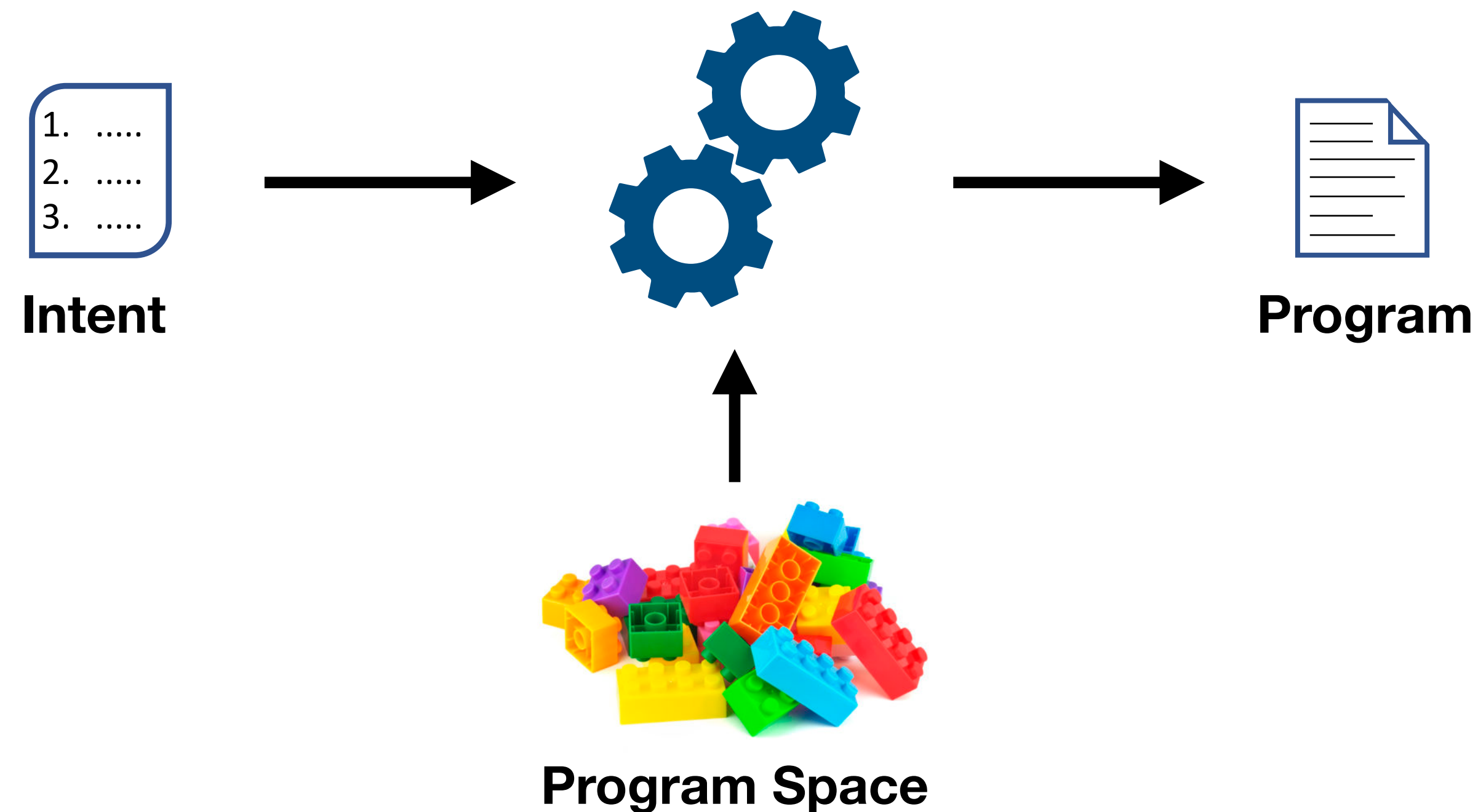
10. Overview of Program Synthesis

Kihong Heo



Program Synthesis 프로그램 합성

- A task of automatically finding a program
 - that satisfy user intent from the underlying program space



History

*“**Instruction tables** will have to be made up by mathematicians with **computing experience** and perhaps a certain **puzzle-solving ability**.
...*

*There need be no real danger of it ever becoming a drudge, for any processes that are quite **mechanical** may be turned over to the **machine itself**.”*

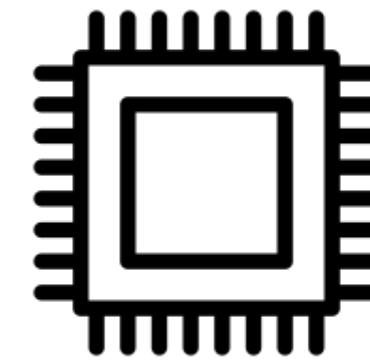
- A. M. Turing, “Proposed Electronic Calculator”, 1946



History of Programming



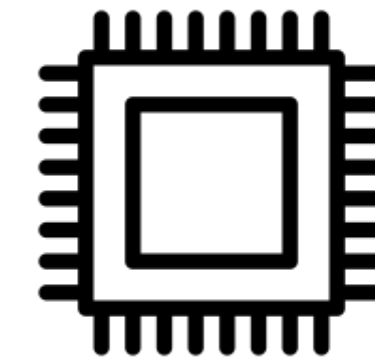
```
quickSort:
.LFB2:
    .cfi_startproc
    endbr64
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    subq    $32, %rsp
    movq    %rdi, -24(%rbp)
    movl    %esi, -28(%rbp)
    movl    %edx, -32(%rbp)
    movl    -28(%rbp), %eax
    cmpl    -32(%rbp), %eax
    jge     .L9
    movl    -32(%rbp), %edx
    movl    -28(%rbp), %ecx
    movq    -24(%rbp), %rax
    movl    %ecx, %esi
    movq    %rax, %rdi
    call    partition
    movl    %eax, -4(%rbp)
    movl    -4(%rbp), %eax
    leal    -1(%rax), %edx
    movl    -28(%rbp), %ecx
    movq    -24(%rbp), %rax
    movl    %ecx, %esi
    movq    %rax, %rdi
    call    quickSort
    movl    -4(%rbp), %eax
    leal    1(%rax), %ecx
    movl    -32(%rbp), %edx
    movq    -24(%rbp), %rax
    movl    %ecx, %esi
    movq    %rax, %rdi
    call    quickSort
.L9:
    nop
    leave
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
```



History of Programming



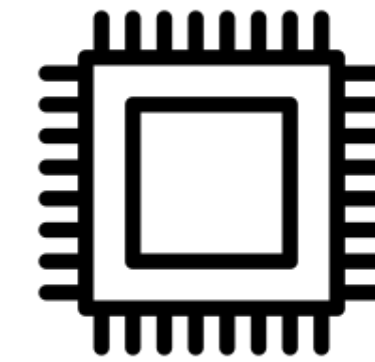
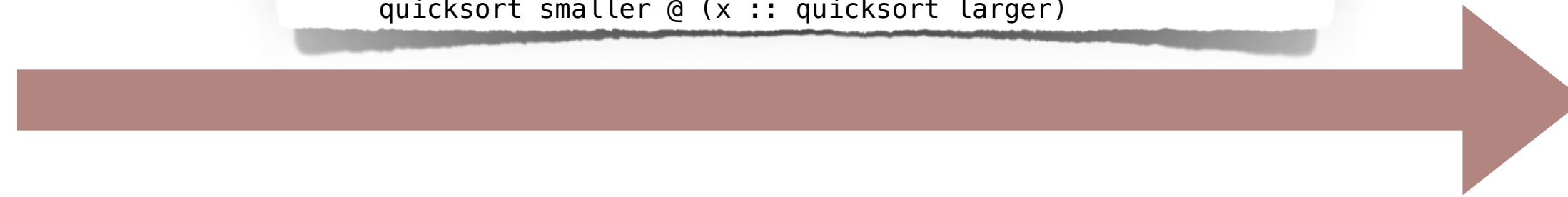
```
void swap(int *a, int *b) {  
    int t = *a;  
    *a = *b;  
    *b = t;  
}  
  
int partition(int array[], int low, int high) {  
    int pivot = array[high];  
    int i = (low - 1);  
    for (int j = low; j < high; j++) {  
        if (array[j] <= pivot) {  
            i++;  
            swap(&array[i], &array[j]);  
        }  
    }  
    swap(&array[i + 1], &array[high]);  
    return (i + 1);  
}  
  
void quickSort(int array[], int low, int high) {  
    if (low < high) {  
        int pi = partition(array, low, high);  
        quickSort(array, low, pi - 1);  
        quickSort(array, pi + 1, high);  
    }  
}
```



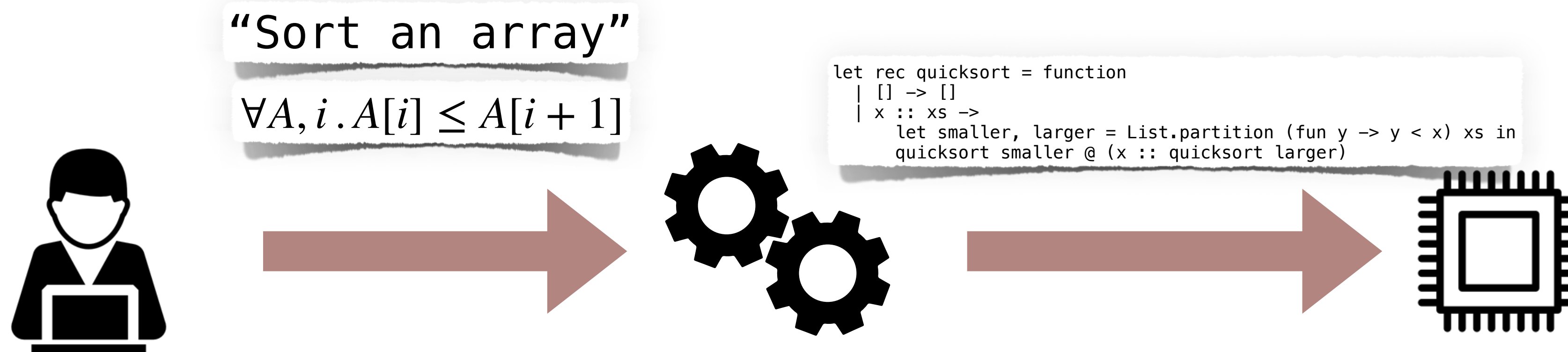
History of Programming



```
let rec quicksort = function
| [] -> []
| x :: xs ->
    let smaller, larger = List.partition (fun y -> y < x) xs in
    quicksort smaller @ (x :: quicksort larger)
```



Future of Programming



Why Program Synthesis?

- Human makes mistakes
- #developers << #programming tasks (feature reqs, bug fixes)
 - E.g., Linux kernel's open bug reports
- Programming for everybody



```
...
hashOut.data = hashes + SSL_MD5_DIGEST_LEN;
hashOut.length = SSL_SHA1_DIGEST_LEN;
if ((err = SSLFreeBuffer(&hashCtx)) != 0)
    goto fail;
if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
    goto fail; /* MISTAKE! THIS LINE SHOULD NOT BE HERE */
if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
    goto fail;

err = sslRawVerify(...);
...
fail:
...
return err;
```



goto fail, 2014
MacOS / iOS
CVE-2014-1266

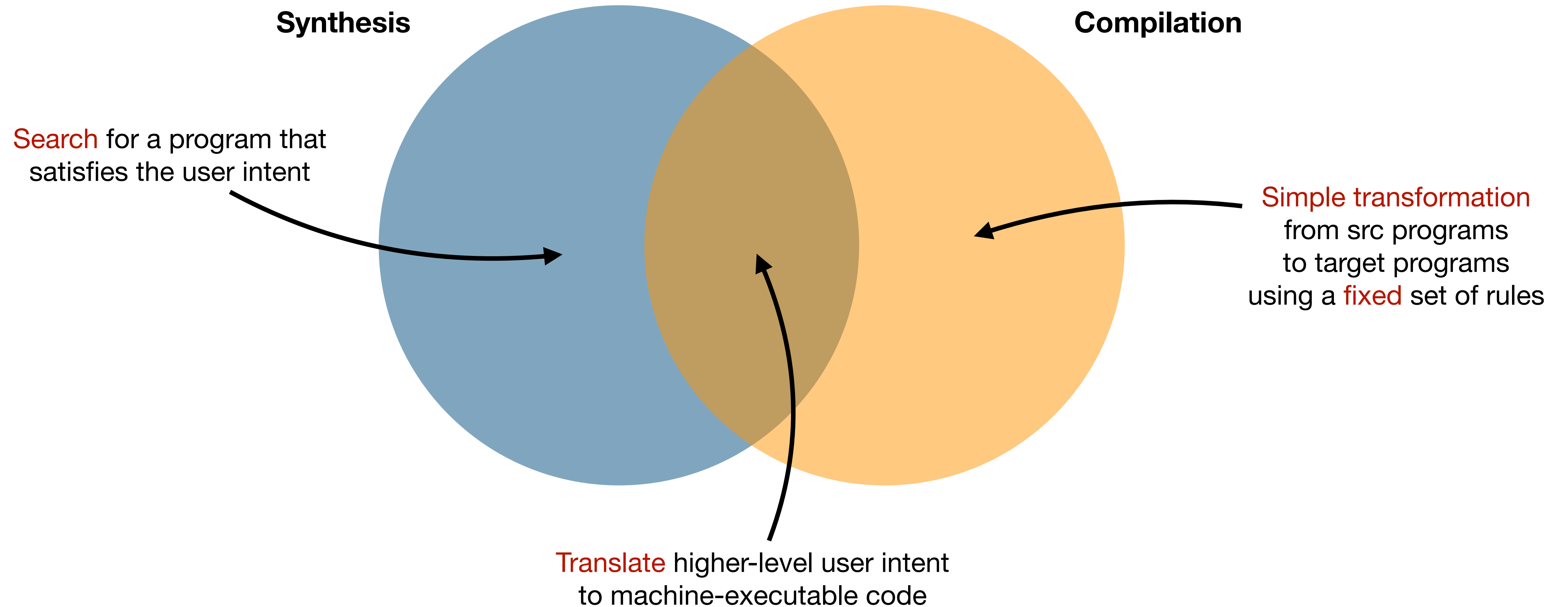
Why Program Synthesis in the Age of AI? (1)

- Why writing programs rather than directly solving with LLM?
 - “How many Rs are there in strawberry” vs “Write a program that counts #Rs in the input”
 - “Sort this list: [1;4;6;5]” vs “Write a quick sort”
- Interpretability: code vs neural net weights
- Verifiability: program verification vs ???
- Modifiability: code patch vs re-training
- Computation cost: GPU vs CPU

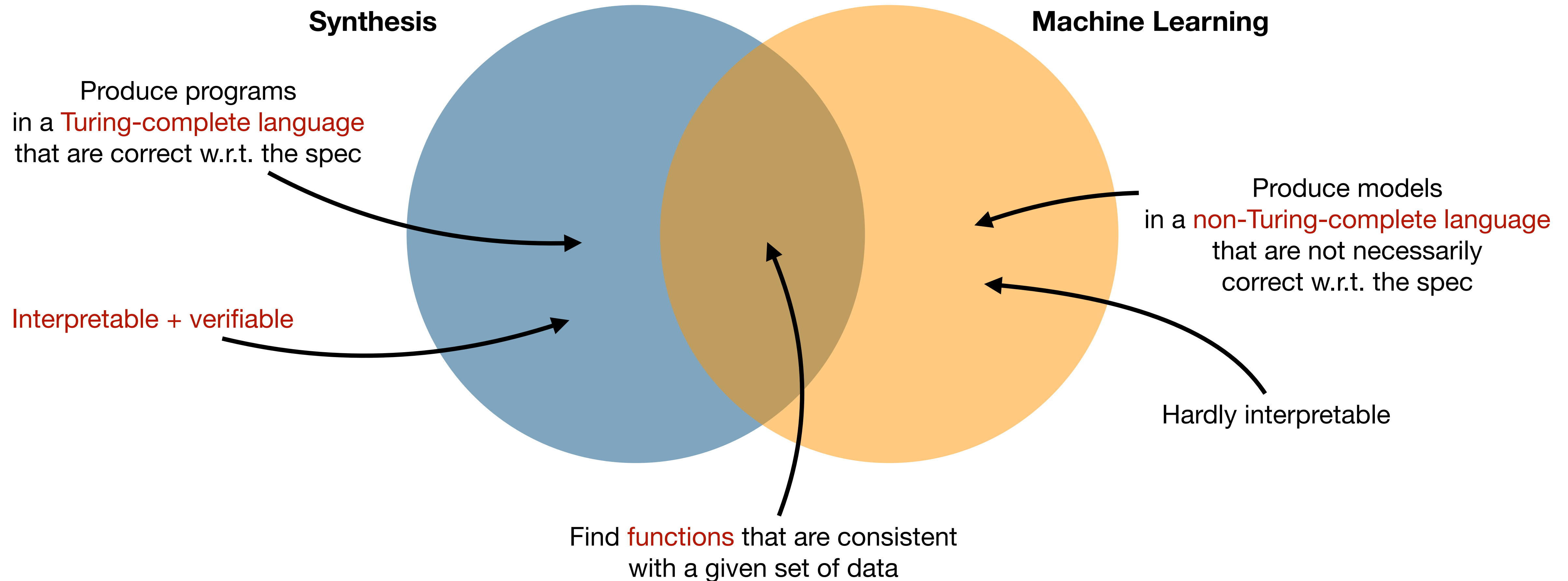
Why Program Synthesis in the Age of AI? (2)

- Why symbolic and search-based synthesis?
- Complementary: logic-based and probability-based
- Idea rather than technique:
 - “Why do we learn quicksort? Dijkstra algorithm?”
 - Application: code, spec, proof, etc
- Examples:
 - Expecto: <https://prosys.kaist.ac.kr/expecto/>, <https://github.com/prosyslab/pl-wiki/wiki/Expecto>

Synthesis vs Compilation

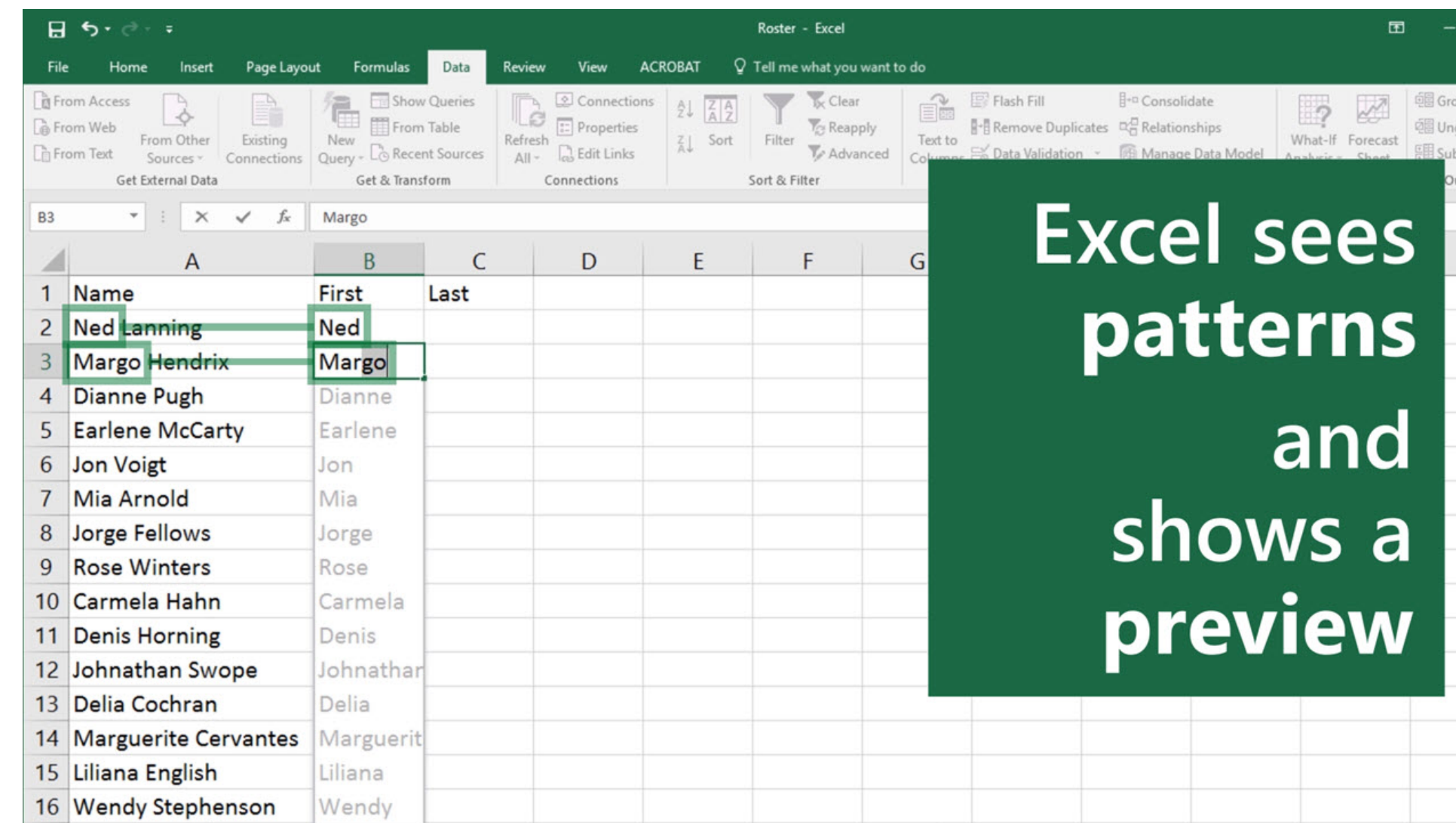


Synthesis vs Machine Learning



Application: End-user Programming (1)

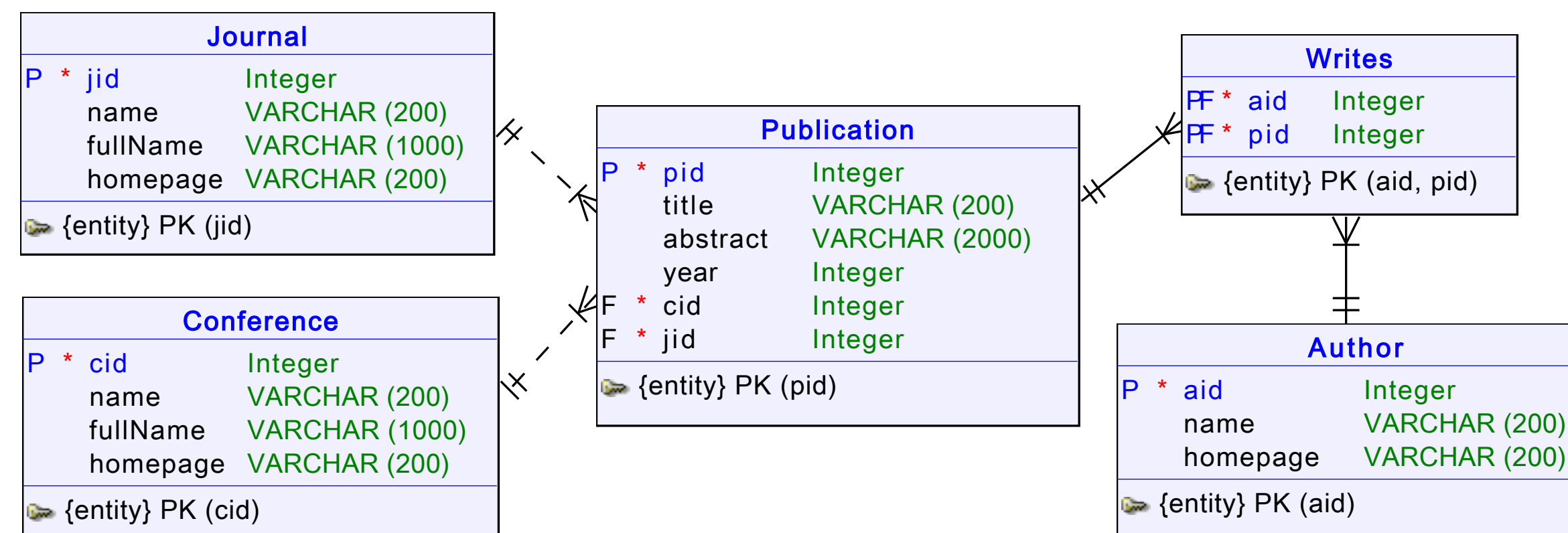
- String transformation (e.g., MS Excel's FlashFill)



Automating String Processing in Spreadsheets using Input-Output Examples, POPL'11

Application: End-user Programming (2)

- SQL query synthesis



“Find the number of papers in OOPSLA 2020”

```
SELECT count(Publication.pid)
FROM Publication JOIN Conference ON Publication.cid = Conference.cid
WHERE Conference.name = "OOPSLA" AND Publication.year = 2010
```

SQLizer: Query Synthesis from Natural Language, OOPSLA'17

Application: Super Optimization

- Montgomery multiplication kernel from the OpenSSL RSA library

```
1 # gcc -O3                               1 # STOKE
2                                           2
3 .L0:                                     3 .L0:
4     movq rsi, r9                         4     shlq 32, rcx
5     movl ecx, ecx                       5     movl edx, edx
6     shrq 32, rsi                        6     xorq rdx, rcx
7     andl 0xffffffff, r9d                7     movq rcx, rax
8     movq rcx, rax                       8     mulq rsi
9     movl edx, edx                       9     addq r8, rdi
10    imulq r9, rax                       10    adcq 0, rdx
11    imulq rdx, r9                       11    addq rdi, rax
12    imulq rsi, rdx                       12    adcq 0, rdx
13    imulq rsi, rcx                       13    movq rdx, r8
14    addq rdx, rax                       14    movq rax, rdi
15    jae .L2
16    movabsq 0x100000000, rdx
17    addq rdx, rcx
18 .L2:
19    movq rax, rsi
20    movq rax, rdx
21    shrq 32, rsi
22    salq 32, rdx
23    addq rsi, rcx
24    addq r9, rdx
25    adcq 0, rcx
26    addq r8, rdx
27    adcq 0, rcx
28    addq rdi, rdx
29    adcq 0, rcx
30    movq rcx, r8
31    movq rdx, rdi
```

Stochastic Super Optimization, ASPLOS'13

Application: Reverse Engineering

- Binary lifting



```

#include <Halide.h>
#include <vector>
using namespace std;
using namespace Halide;

int main() {
    Var x_0;
    Var x_1;
    ImageParam input;
    Func output_1;
    output_1(x_0, x_1) =
        cast<uint8_t>(
            (2 * cast<uint
                cast<uint
            >> cast<uint.
    vector<Argument>
    args.push_back(i
    output_1.compile
    return 0;
}

```

```
>
d;
slide;

_1(UInt(8),2);

) =
(((2+
32_t>(input_1(x_0
32_t>(input_1(x_0
32_t>(input_1(x_0
32_t>(2))) & 255)
args;
input_1);
_to_file("halide_
```

```

+1, x_1+1)) ) +
, x_1+1)) +
+2, x_1+1)) )
);

out_0",args);

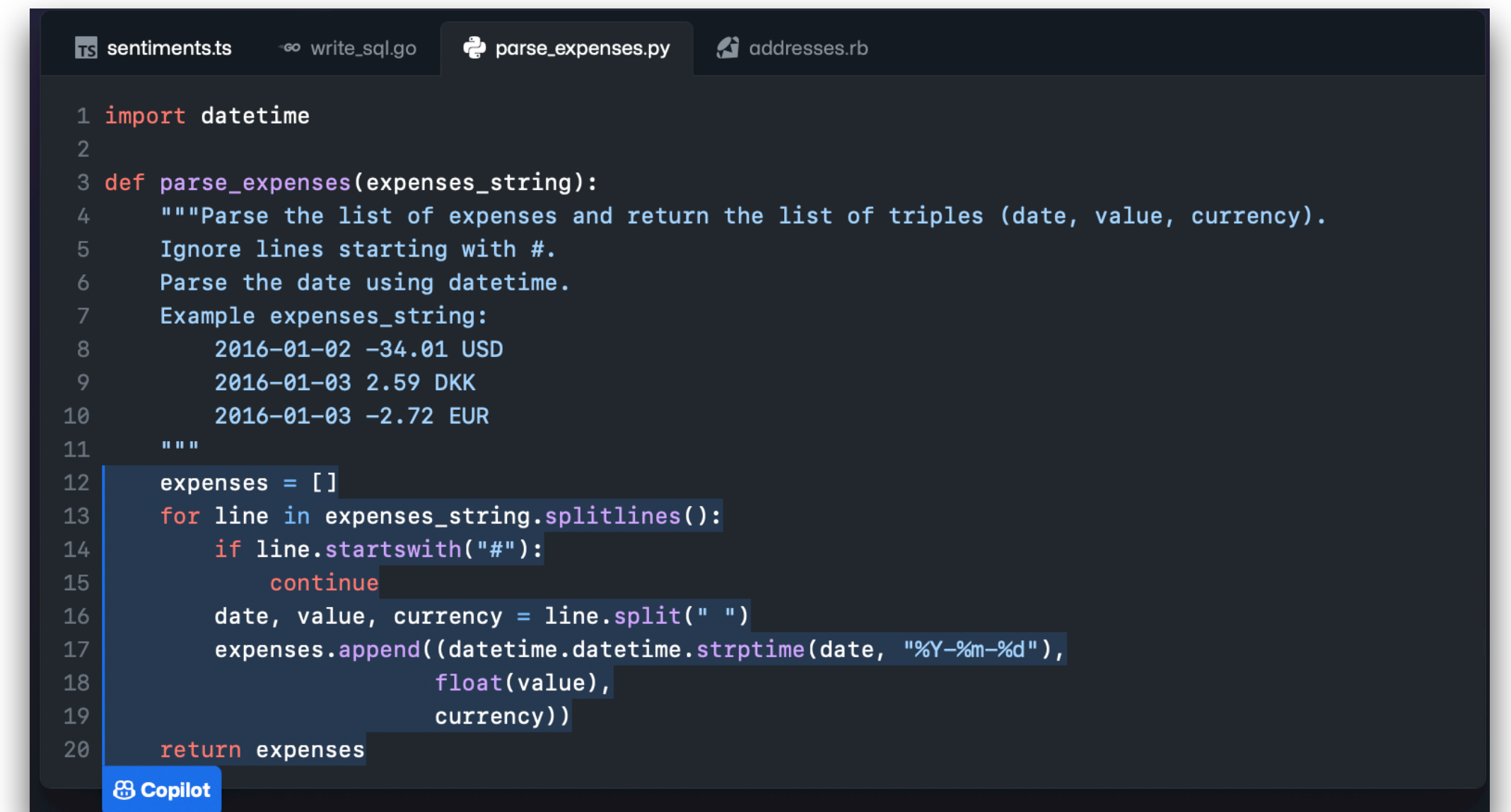
```

```
#include <Halide.h>
#include <vector>
using namespace std;
using namespace Halide;

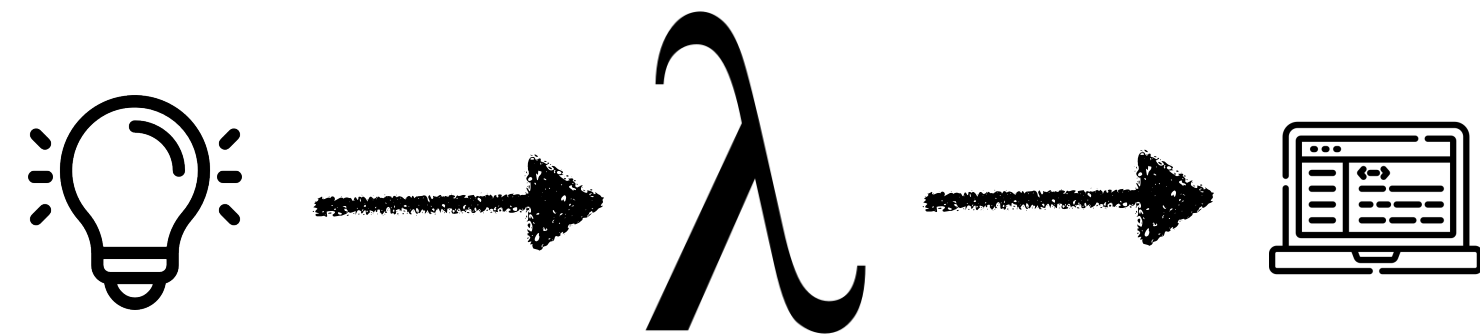
int main(){
    Var x_0;
    Var x_1;
    ImageParam input_1(UInt(8),2);
    Func output_1;
    output_1(x_0,x_1) =
        cast<uint8_t>((((2+
            (2*cast<uint32_t>(input_1(x_0+1,x_1+1))) +
            cast<uint32_t>(input_1(x_0, x_1+1))) +
            cast<uint32_t>(input_1(x_0+2,x_1+1)))
            >> cast<uint32_t>(2))) & 255));
    vector<Argument> args;
    args.push_back(input_1);
    output_1.compile_to_file("halide_out_0",args);
    return 0;
}
```

Helium: Lifting High-Performance Stencil Kernels from Stripped x86 Binaries to Halide DSL Code, PLDI'15

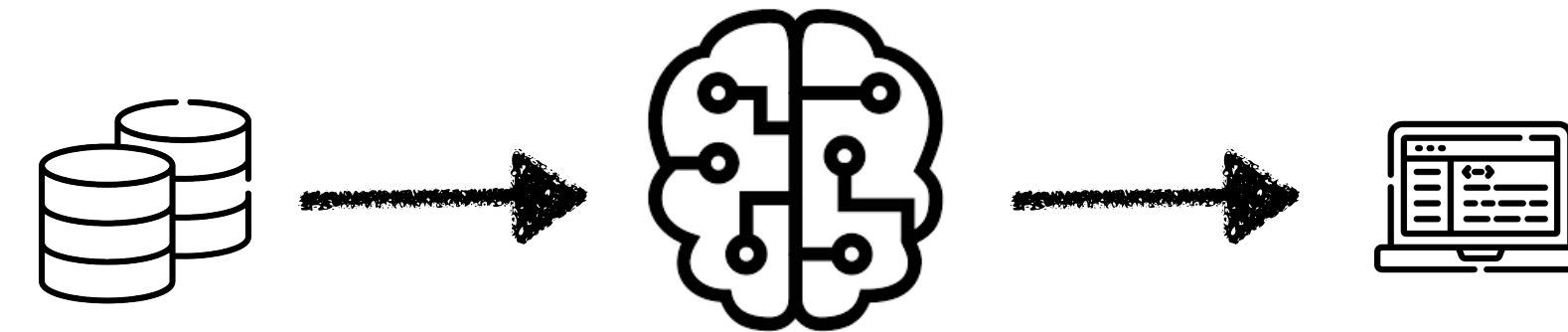
Application: Code Suggestion/Generation



Two Directions: PL vs ML



- Semantics (logical reasoning)
- Correctness guaranteed
- Not very scalable
- Search algorithms, constraint solving
- Example: Excel FlashFill

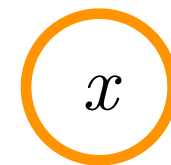


- Likelihood (probabilistic reasoning)
- Correctness not guaranteed
- Very scalable
- Probabilistic inference
- Example: GPT, Claude

Challenge 1: Search Space (1)

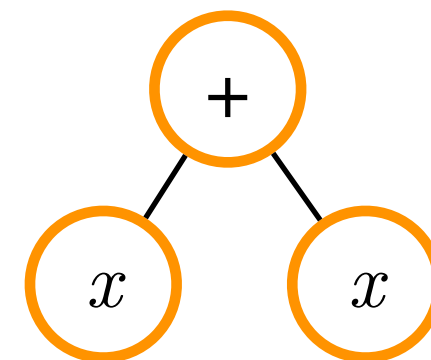
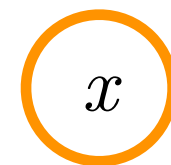
$$S \rightarrow x \mid S + S$$

depth ≤ 1



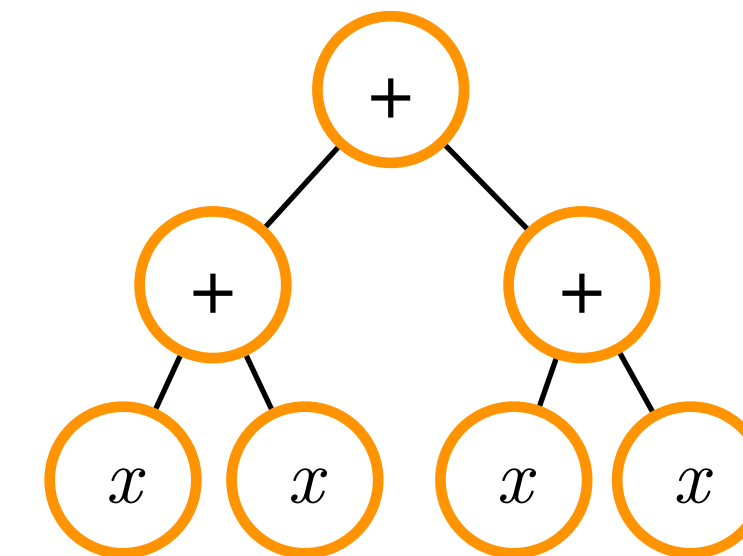
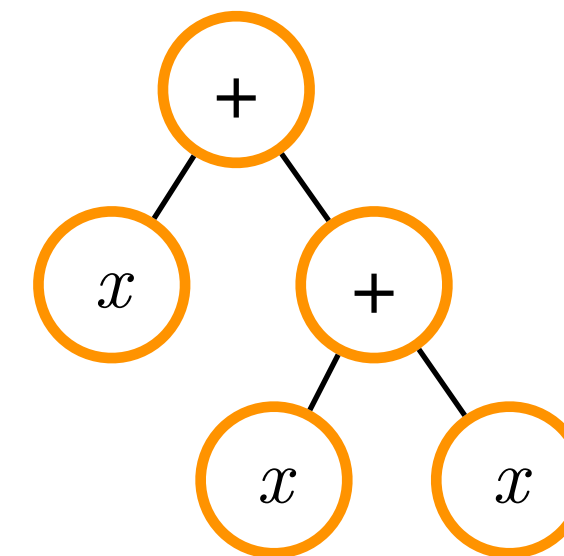
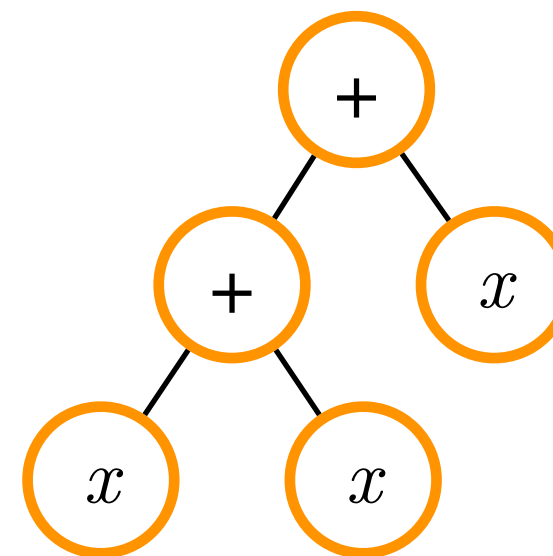
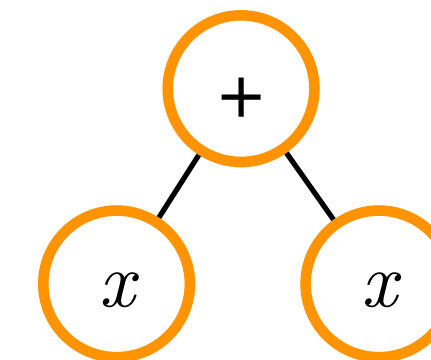
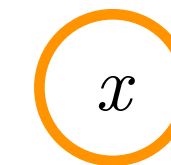
$$N(1) = 1$$

depth ≤ 2



$$N(2) = 2$$

depth ≤ 3



$$N(3) = 5$$

$$N(d) = 1 + N(d - 1)^2$$

Challenge 1: Search Space (2)

$$S \rightarrow x \mid S + S$$

$$N(d) = 1 + N(d - 1)^2$$

$$N(1) = 1$$

$$N(2) = 2$$

$$N(3) = 5$$

$$N(4) = 26$$

$$N(5) = 677$$

$$N(6) = 458330$$

$$N(7) = 210066388901$$

$$N(8) = 44127887745906175987802$$

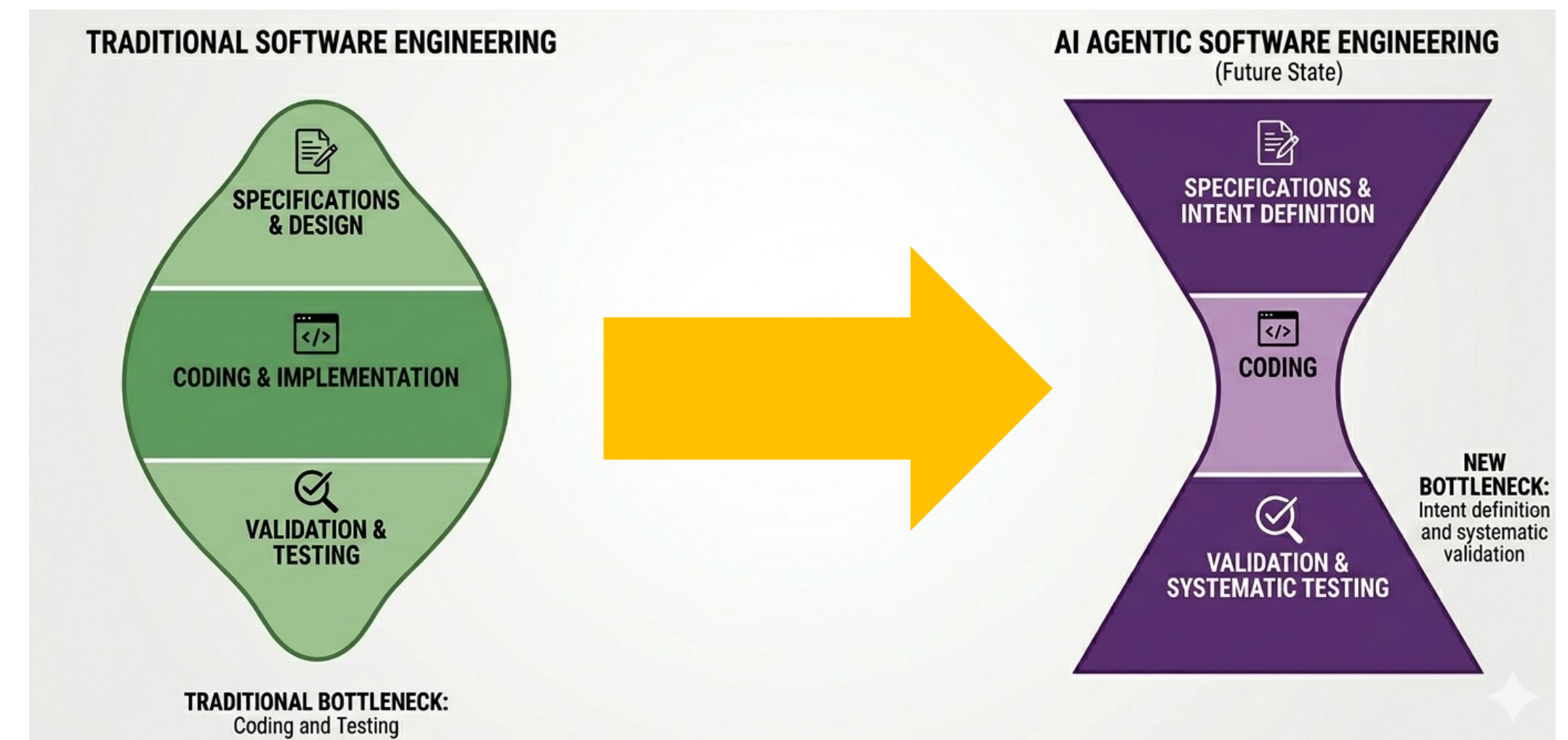
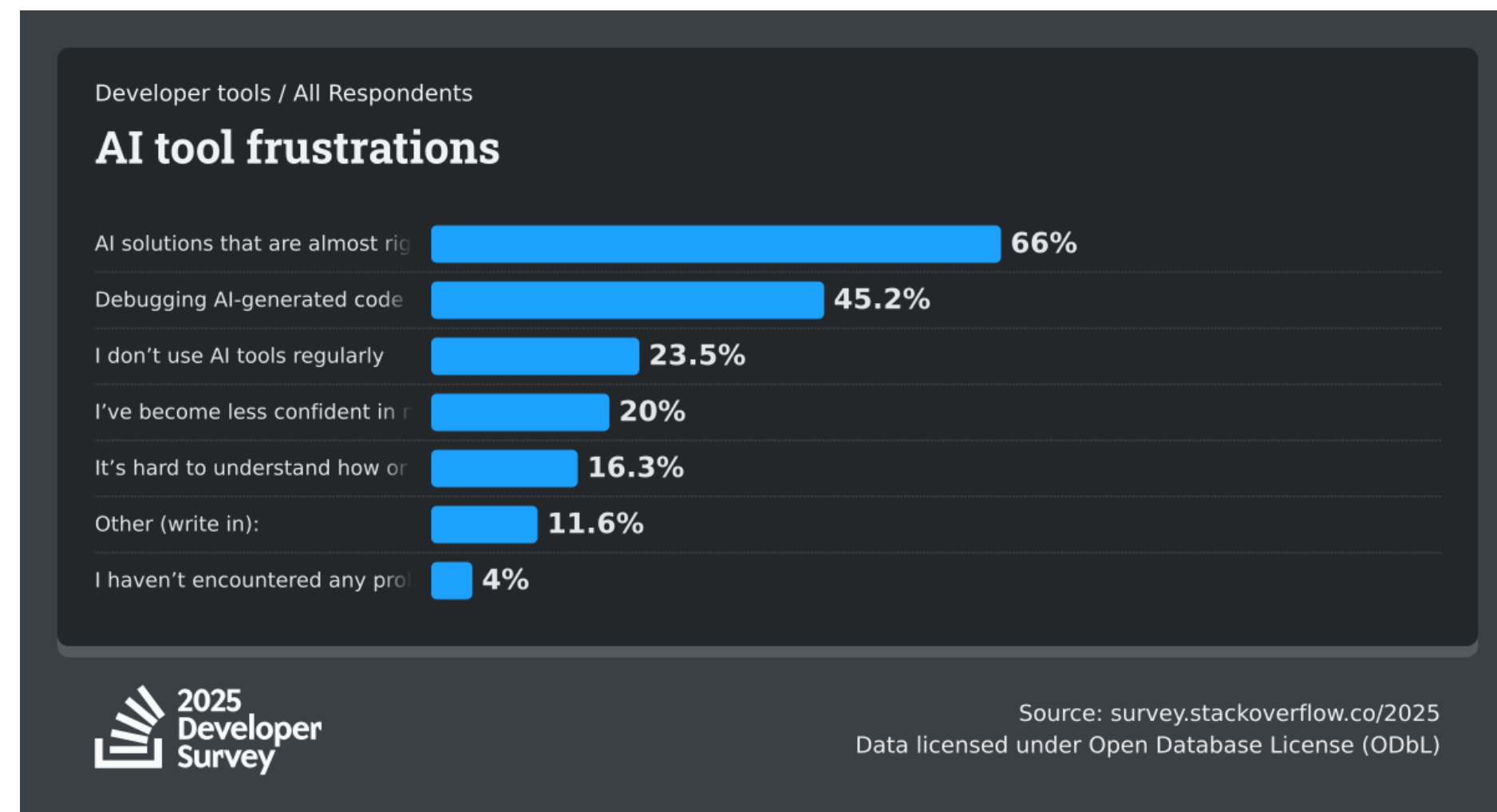
$$N(9) = 1947270476915296449559703445493848930452791205$$

$$N(10) = 3791862310265926082868235028027893277370233152247388584761734150717768254410341175325352026$$



*Examples from Nadia Polikarpova's slides

Challenge 2: Correctness (1)



- A lot of code is generated that “looks correct”*
- Harder to detect than obviously incorrect code
- Verification cost $\gg$ generation cost**
- Increased safety and security risks

*2025 Stack Overflow developer survey

**Miryung Kim, “Happiness U-Curve”, PLDI 2026 Keynote

Challenge 2: Correctness (2)

- Recurring SW vulnerabilities by human developers
- Example: gimp-2.6.7 (CVE-2009-1570)

```
long ToL (char *pbuffer) { return (puffer[0] | puffer[1]<<8 | puffer[2]<<16 | puffer[3]<<24); } 3
short ToS (char *pbuffer) { return ((short)(puffer[0] | puffer[1]<<8)); }
gint32 ReadBMP (gchar *name, GError **error) { 1
    if (fread(buffer, Bitmap_File_Head.biSize - 4, fd) != 0)
        FATALP ("BMP: Error reading BMP file header #3");
    Bitmap_Head.biWidth = ToL (&buffer[0x00]); 2
    Bitmap_Head.biBitCnt = ToS (&buffer[0x0A]);

    rowbytes = ((Bitmap_Head.biWidth * Bitmap_Head.biBitCnt - 1) / 32) * 4 + 4; 4
    image_ID = ReadImage (rowbytes); 5
    ...
}

gint32 ReadImage (int rowbytes) {
    buffer = malloc(rowbytes); 6 // malloc with overflowed size
    ...
}
```

Challenge 2: Correctness (3)

- Recurring SW vulnerabilities by human developers
- Example: sam2p-0.49.4 (CVE-2017-1663)

```
long ToL (char *pbuffer) { return (puffer[0] | puffer[1]<<8 | puffer[2]<<16 | puffer[3]<<24); } 3

short ToS (char *pbuffer) { return ((short)(puffer[0] | puffer[1]<<8)); }

bitmap_type bmp_load_image (FILE* filename) { 1
    if (fread(buffer, Bitmap_File_Head.biSize - 4, fd) != 0)
        FATALP ("BMP: Error reading BMP file header #3");
    Bitmap_Head.biWidth = ToL (&buffer[0x00]); 2
    Bitmap_Head.biBitCnt = ToS (&buffer[0x0A]);

    rowbytes = ((Bitmap_Head.biWidth * Bitmap_Head.biBitCnt - 1) / 32) * 4 + 4; 4
    image.bitmap = ReadImage (rowbytes); 5
    ...
}

unsigned char* ReadImage (int rowbytes) { 6
    unsigned char *buffer = (unsigned char*) new char[rowbytes]; // malloc with overflowed size
    ...
}
```


Challenge 2: Correctness (3)

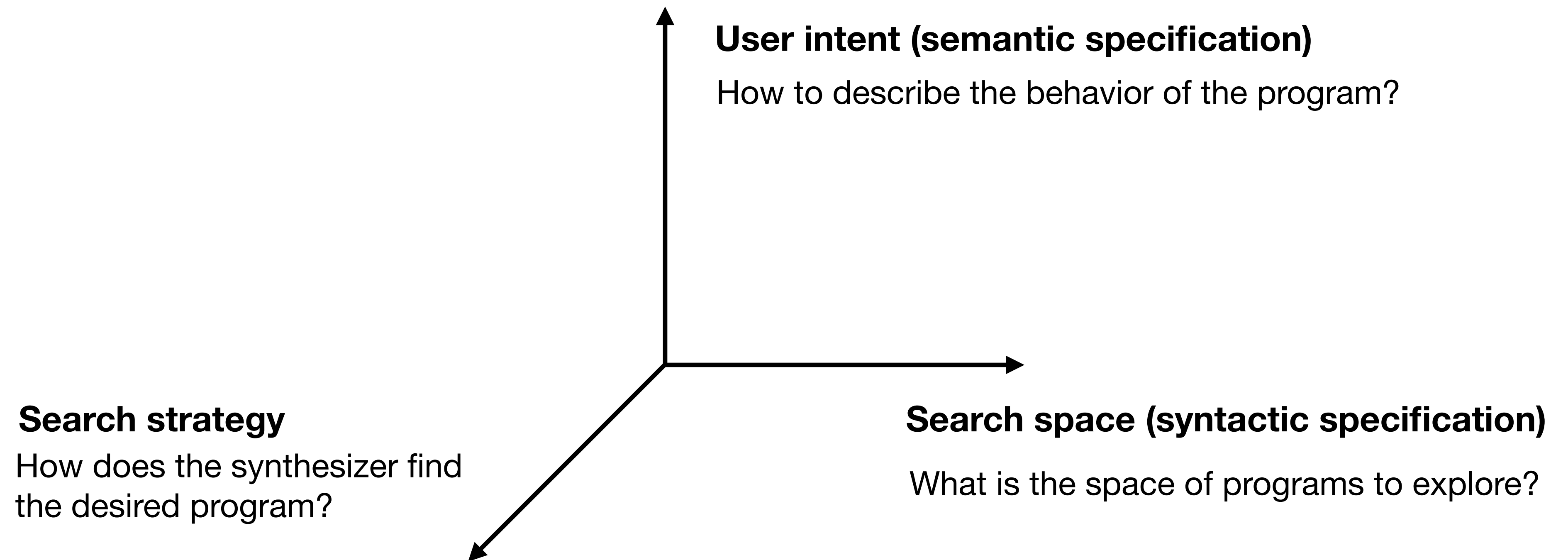
- Recurring SW vulnerabilities by **AI developers**
- Example (feat. Copilot):

```
int toLong(char *buffer) {  
    return (buffer[0]) | (buffer[1] << 8) | (buffer[2] << 16) | (buffer[3] << 24);  
}  
  
int f(char *name) {  
    int width, height, area;  
    char buffer[10];  
    FILE *fd = fopen(name, "rb");  
    fread(buffer, 10, 1, fd);  
    fclose(fd);  
  
    // Copilot, fill in the blank!  
    width = toLong(buffer + 18);  
    height = toLong(buffer + 22);  
    area = width * height;  
    return area;
```

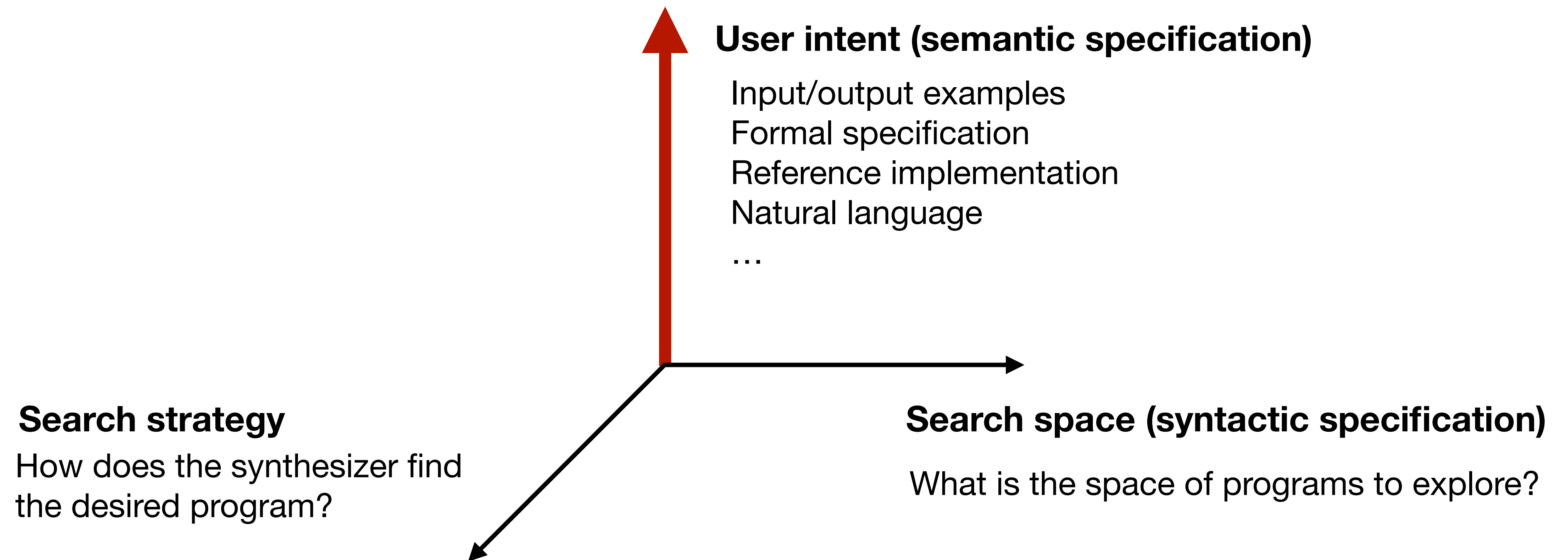
Grand Challenge

- How to achieve **scalable** and **correct** program synthesizers?
 - Logical inference and search algorithms are enough?
 - Large language models are enough?
- In this semester
 - “PL-based” techniques: synthesize a program that satisfies a given specification
 - Neuro-symbolic techniques: guide LLM to satisfy given constraints
- After this semester
 - Hope you develop more powerful synthesizers combining PL and ML

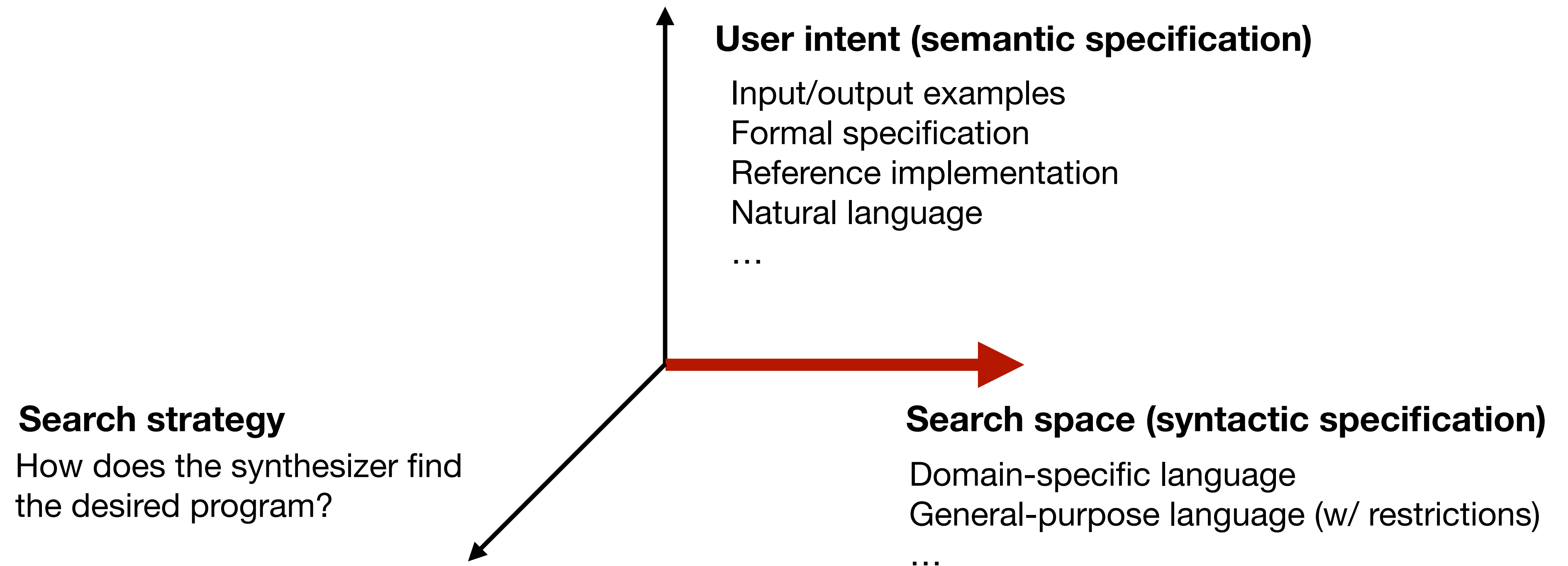
Dimensions in Program Synthesis



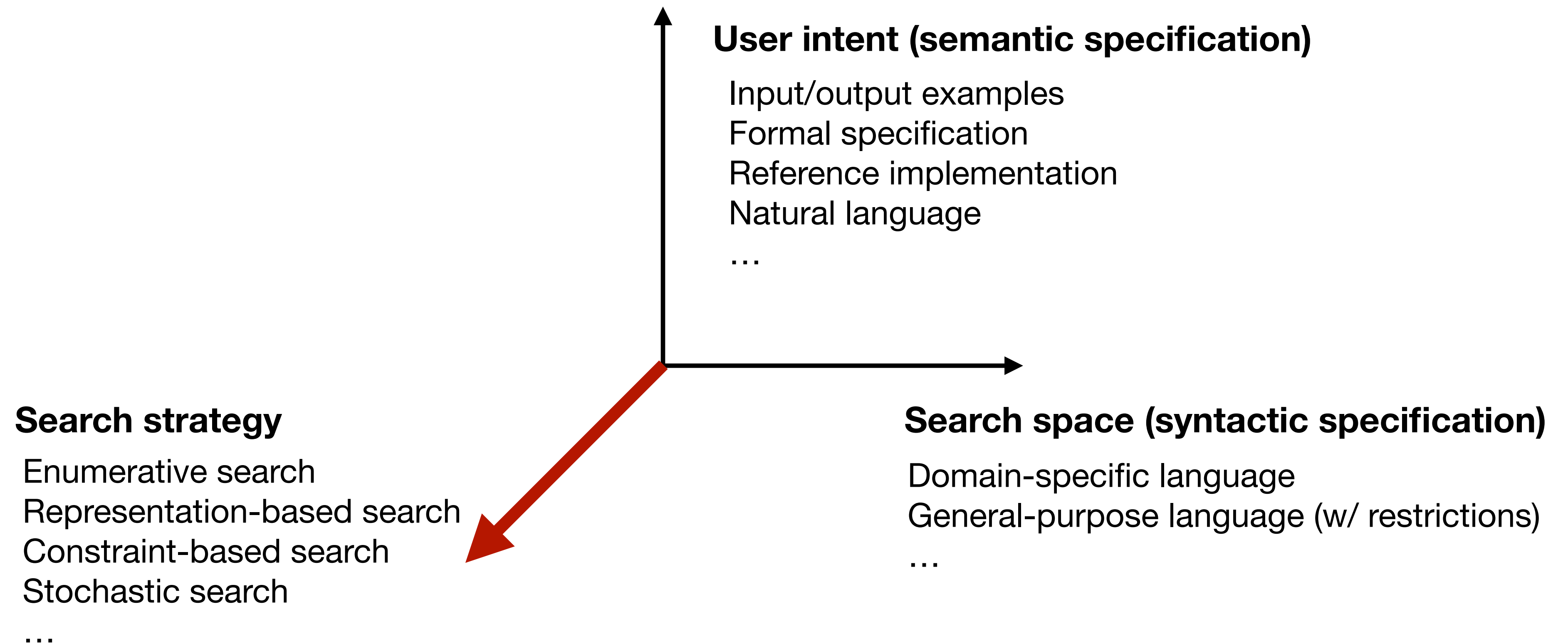
Dimensions in Program Synthesis



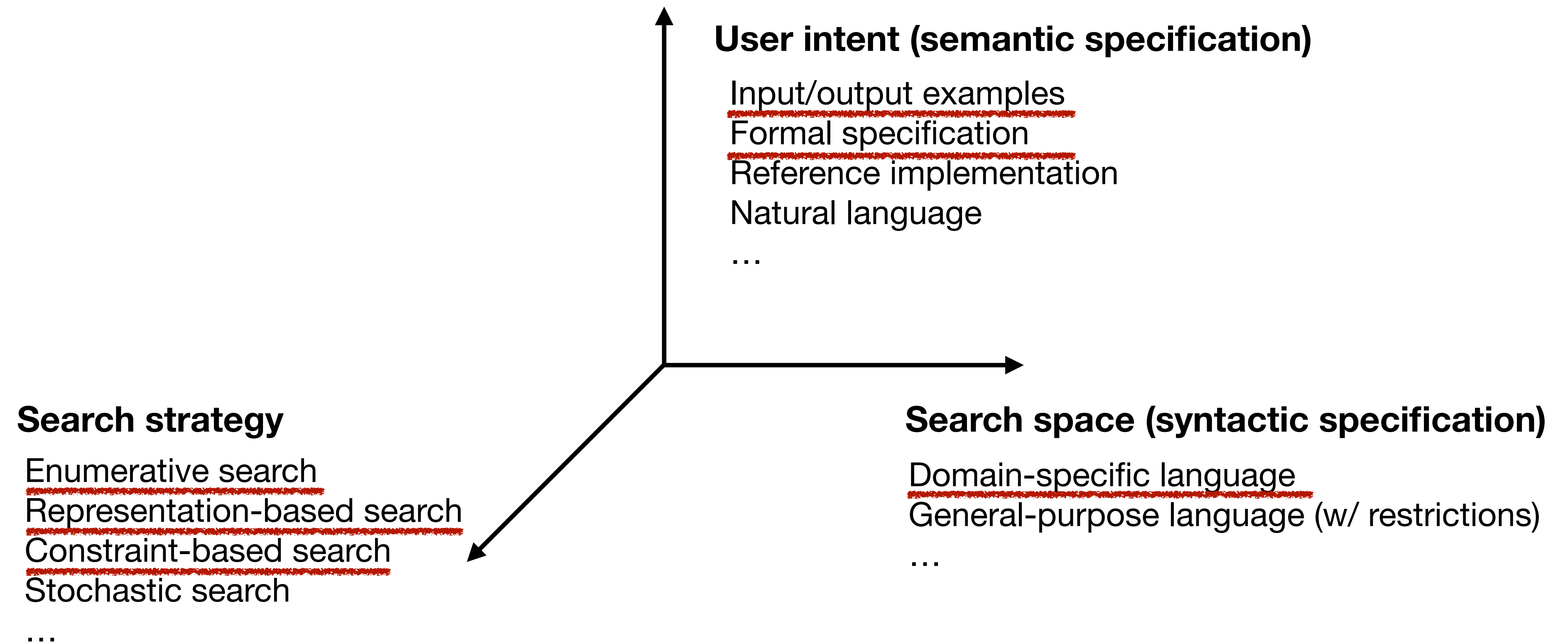
Dimensions in Program Synthesis



Dimensions in Program Synthesis



Dimensions in Program Synthesis



***This course**

Summary

- Program synthesis: **automated programming** systems (the holy grail of computer science)
- Principles: syntactic / semantics specification, search strategy
- Applications: software engineering, compiler, security, etc
- Challenges: scalability, correctness, expressiveness, etc
- Opportunities:
 - Neurosymbolic AI
 - $AGI = \text{System 1} + \text{System 2}$